

Escuela de Ingeniería en Computadores

Curso: Bases de Datos (CE3101)

II Semestre 2026

Proyecto:

Smart City

Entrega:

Sprint1 PostgreSQL y MongoDB

Equipo de trabajo

República Independiente de Cartago (RIC)

Rodrigo Arce Bastos 2024115582

Dylan Elizondo Picado 2023086320

Kevin Josué Ruiz Rodríguez 2018170538

Allan Ruben Prieto Badilla 2021032988

Profesor:

Marco Rivera Menese

Fecha de Entrega:

28/10/2026

Objetivo del sprint

El objetivo de este sprint es diseñar, implementar y validar las bases de datos PostgreSQL y MongoDB de SmartCity Analytics, utilizando datos de tráfico vehicular y ocupación de parqueos. Se busca organizar e integrar ambas fuentes mediante estructuras que permitan identificar la procedencia de cada registro, conservar sus fechas y ubicaciones, evitar duplicados y mantener la integridad de la información. PostgreSQL almacenará los catálogos y los datos estructurados, mientras que MongoDB almacenará las lecturas y la información semiestructurada asociada. Mediante consultas SQL y agregaciones en MongoDB, se analizarán la ocupación y disponibilidad de los parqueos, así como el comportamiento del tráfico según las variables disponibles, las zonas y los períodos registrados. Al finalizar el sprint, se espera contar con modelos documentados, scripts reproducibles y pruebas que demuestren el funcionamiento de ambas bases, estableciendo una base confiable para las siguientes etapas de procesamiento y visualización del proyecto.

Fuente de datos y su procedencia

Para el desarrollo de este sprint se seleccionaron dos conjuntos de datos relacionados con la movilidad urbana: uno sobre ocupación de parqueos y otro sobre tráfico vehicular. Ambos fueron localizados mediante **IoTDataset**, que funciona como catálogo y enlaza a sus respectivas publicaciones en **Kaggle**. Se distingue entre el sitio donde se encontraron los datos y la publicación desde la cual se obtendrán los archivos utilizados en el proyecto.

Fuente	Información utilizada	Referencia
Parking Birmingham	Identificador del parqueo, capacidad, ocupación y fecha.	Fuente de parqueos
Urban Traffic Data	Información de tráfico por ciudad, ruta y período, según el archivo utilizado.	Fuente de tráfico

Diseño y arquitectura de las bases de datos

Distribución de responsabilidades

Componente	Responsabilidad
Proceso de carga	Leer los archivos, transformar campos, resolver identificadores, validar y registrar errores.

PostgreSQL	Mantener catálogos, relaciones y observaciones estructuradas mediante tablas y restricciones.
MongoDB	Almacenar documentos de lectura con contexto y mediciones, además de eventos, alertas e incidentes.

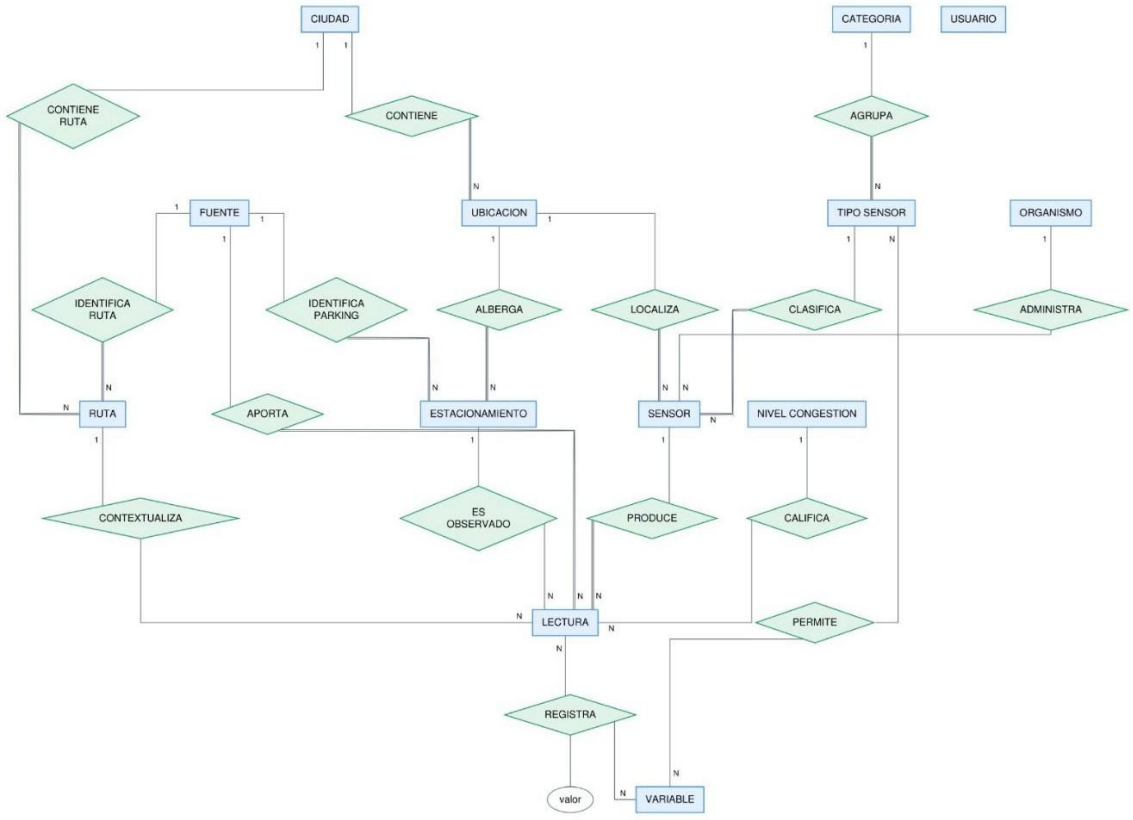
Los identificadores compartidos permiten mantener la correspondencia entre ambos motores. El campo id_documento_mongo de PostgreSQL conserva la referencia al documento de MongoDB; esta relación debe ser comprobada por el proceso de integración.

Diseño de PostgreSQL

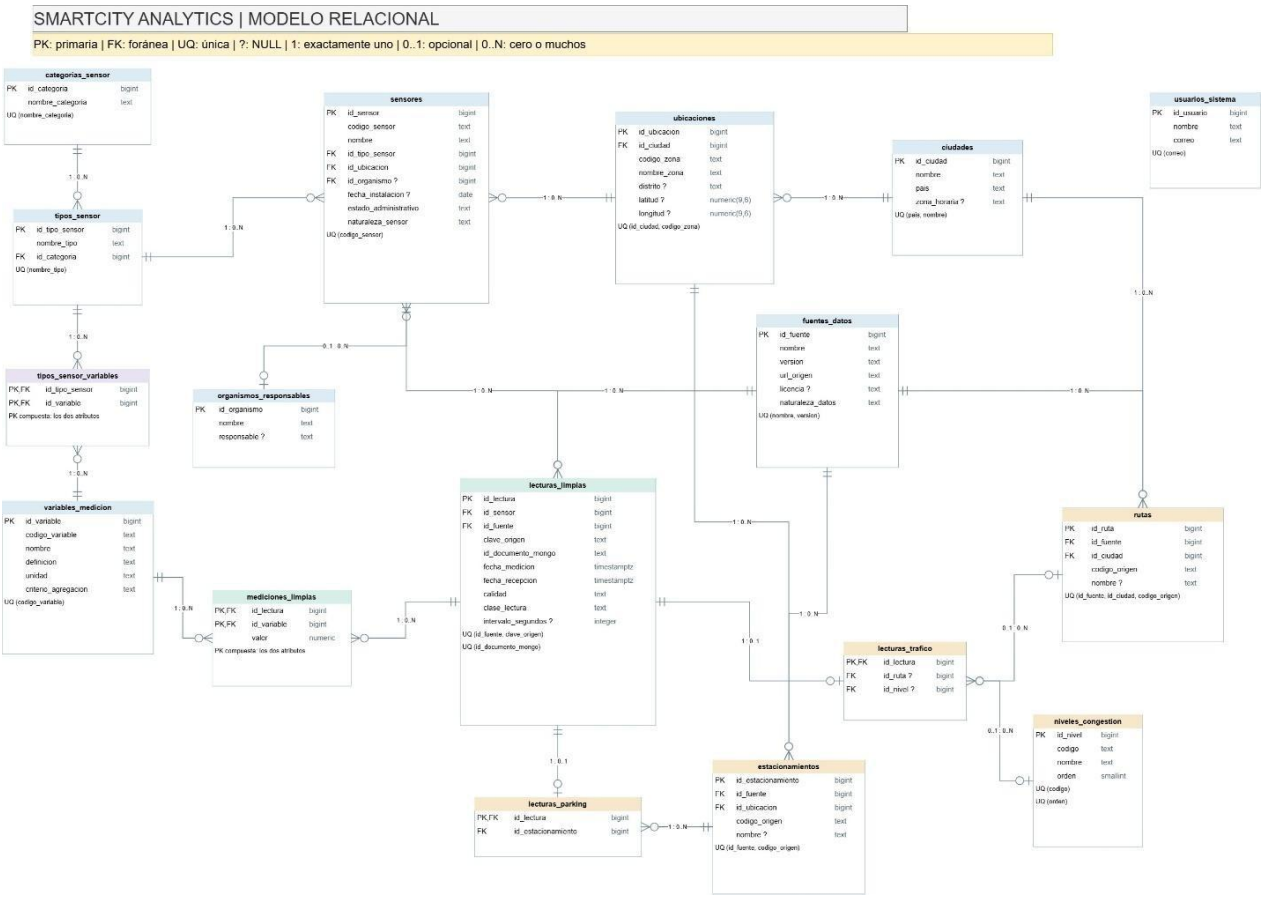
El modelo organiza la información en 17 tablas. Se distinguen los catálogos, las lecturas, las mediciones y los contextos de tráfico y parqueo. Las claves primarias identifican los registros y las claves foráneas mantienen las referencias entre tablas.

La separación entre lectura y medición permite asociar varias variables con una observación. Los catálogos evitan repetir descripciones y facilitan mantener el modelo normalizado.

Modelo Conceptual



Modelo Relacional



Matriz de participación y cardinalidad

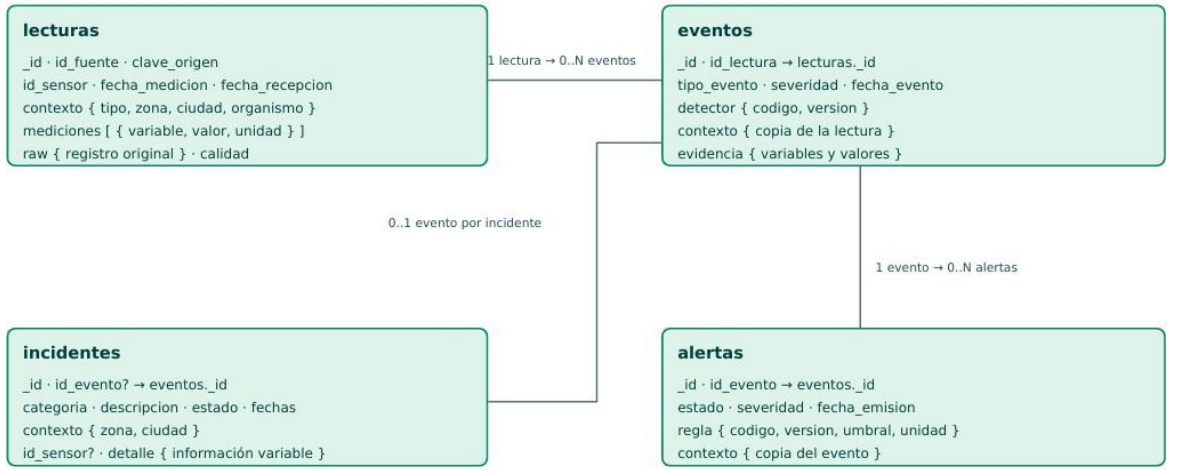
Entidad A	Relación	Entidad B	A (mín,máx)	B (mín,máx)
CATEGORIA	AGRUPA	TIPO_SENSOR	(0,N)	(1,1)
TIPO_SENSOR	CLASIFICA	SENSOR	(0,N)	(1,1)
ORGANISMO	ADMINISTRA	SENSOR	(0,N)	(0,1)
CIUDAD	CONTIENE	UBICACION	(0,N)	(1,1)
UBICACION	LOCALIZA	SENSOR	(0,N)	(1,1)
TIPO_SENSOR	PERMITE	VARIABLE	(0,N)	(0,N)
SENSOR	PRODUCE	LECTURA	(0,N)	(1,1)
FUENTE	APORTA	LECTURA	(0,N)	(1,1)
LECTURA	REGISTRA	VARIABLE	(0,N)	(0,N)
UBICACION	ALBERGA	ESTACIONAMIENTO	(0,N)	(1,1)
CIUDAD	CONTIENE_RUTA	RUTA	(0,N)	(1,1)
FUENTE	IDENTIFICA_PARKING	ESTACIONAMIENTO	(0,N)	(1,1)
FUENTE	IDENTIFICA_RUTA	RUTA	(0,N)	(1,1)
ESTACIONAMIENTO	ES_OBSERVADO	LECTURA	(0,N)	(0,1)
RUTA	CONTEXTUALIZA	LECTURA	(0,N)	(0,1)
NIVEL_CONGESTION	CALIFICA	LECTURA	(0,N)	(0,1)

Diseño de MongoDB

Colección	Contenido
lecturas	Observaciones con sensor, fuente, fechas, contexto y mediciones.
eventos	Hechos detectados y vinculados con una lectura.
alertas	Avisos relacionados con eventos y reglas identificadas.
incidentes	Sucesos reportados, con referencias opcionales a sensores y eventos.

Diseño documental

Las cajas son colecciones y objetos embebidos. Las líneas entre colecciones son referencias lógicas, no claves foráneas automáticas.



Los IDs y el contexto se resuelven desde PostgreSQL al ingresar cada documento. No se agregan arrays crecientes de lecturas, eventos o alertas dentro de un sensor.

PostgreSQL: catálogos autoritativos

sensores · tipos · categorías · ubicaciones · ciudades · organismos · fuentes · variables · rutas · estacionamientos

Contrato común e identificadores

Campo	Tipo BSON / regla	Aplicación
_id	string no vacía; clave única e inmutable	Todas. ID estable generado por la ingesta. Se usan prefijos lec:, evt:, ale: e inc:.
schema_version	int = 1	Todas; permite evolucionar el esquema con migraciones explícitas.
id_fuente	long positivo	Todas. Mismo ID del catálogo PostgreSQL; identifica una versión del dataset.
clave_origen	string no vacía	Todas. Clave estable por colección y fuente; incluye tipo de derivación si aplica.
naturaleza	real | sintetica | no_verificada	Todas. Se hereda de la fuente; no se presenta una simulación como observación real.
procedencia	original | derivado | simulado	Todas. Describe cómo se produjo el documento, no la naturaleza de su fuente.
contexto	object	Todas. id_ubicacion e id_ciudad obligatorios. nombre_zona opcional; copia de catálogo.
contexto del sensor	id_tipo_sensor, id_categoria: long; id_organismo?: long	Obligatorio en lecturas/eventos/alertas. En incidentes, solo si existe id_sensor.
id_sensor	long positivo	Obligatorio en lecturas/eventos/alertas; opcional en incidentes.
Fechas y medidas	date; long para IDs; decimal para valores	Fechas UTC; conservar zona de origen documentada. No mezclar strings con números.

Decisiones para estos diseños

Problema	Decisión y justificación
Diferencias entre fuentes	Utilizar un catálogo de variables y contextos específicos para tráfico y parqueos.
Repetición de información	Normalizar los catálogos en PostgreSQL y mantener una copia controlada del contexto en MongoDB.
Posibles duplicados	Identificar cada observación mediante su fuente y clave de origen.
Información desconocida	Permitir referencias opcionales y registrar errores sin inventar datos.
Capacidad y ocupación	Guardar los valores por lectura y calcular el porcentaje cuando la capacidad sea positiva.

Instrucciones de Reproducción y Despliegue del Clúster

La arquitectura de SmartCity Analytics está completamente contenerizada mediante Docker Compose y VS Code Dev Containers, orquestando seis servicios integrados: PostgreSQL 15, MongoDB 6.0, Apache Hadoop HDFS (NameNode y DataNode), Apache Spark 3.5.1, pgAdmin 4 y Mongo Express. Para la guía paso a paso de requisitos del sistema, configuración de Docker Desktop, integración WSL2 y variables de entorno, referirse a la documentación detallada disponible en el archivo README.md de la raíz del repositorio oficial en GitHub (https://github.com/Allan19Prieto/smartcity_project).

Para reproducir y validar de forma automatizada todo el flujo del Sprint 1 (creación del esquema DDL en 3FN, carga de datasets reales CSV de 36.019 registros, generación de 500.000 lecturas sintéticas en ambas bases y verificación de las consultas analíticas obligatorias), se ejecuta en una sola instrucción desde la terminal del contenedor:

```
python sprint1.py
# Para reejecutar consultas sin repetir la carga de datos masiva:
python sprint1.py --skip-load
```

Comparación Tecnológica Crítica: PostgreSQL vs MongoDB

El diseño del proyecto SmartCity Analytics demuestra empíricamente que no existe un único motor de base de datos óptimo para todos los requerimientos urbanos. Se implementó una arquitectura de persistencia políglota donde PostgreSQL gestiona los datos maestros estructurados con integridad transaccional estricta (3FN), mientras que MongoDB almacena la telemetría masiva semiestructurada y series temporales de sensores IoT.

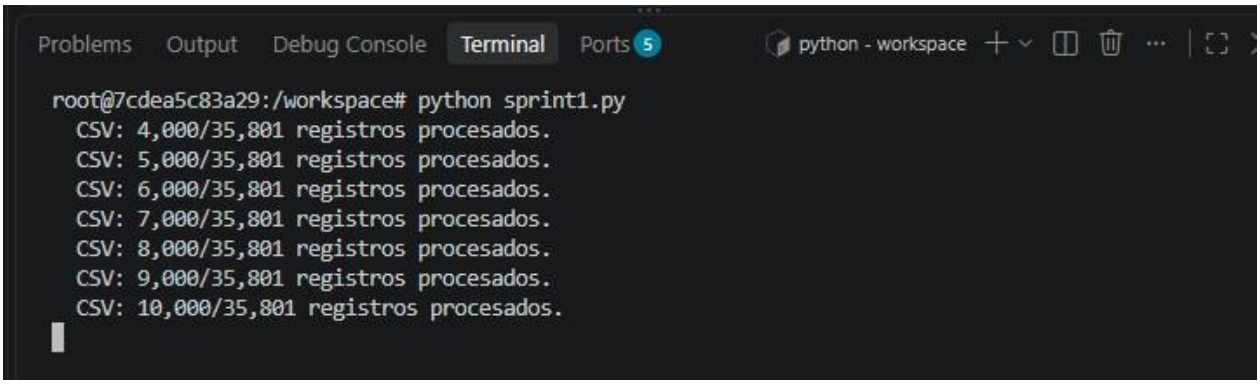
Dimensión Técnica	PostgreSQL 15 (Relacional)	MongoDB 6.0 (Documental NoSQL)	Análisis Crítico y Decisión
Paradigma y Modelo	Relacional (Álgebra de Codd). Esquema tabular rígido con tipos predefinidos.	Documental BSON. Esquema flexible y dinámico validado con $jsonSchema.	PostgreSQL evita anomalías en catálogos maestros. MongoDB permite que lecturas de parking y tráfico coexistan sin columnas dispersas ni NULLs.
Normalización	Tercera Forma Normal (3FN)	Denormalización controlada.	La 3FN en Postgres optimiza

	estricta. Cero redundancia; normalizado en 17 tablas.	Documentos autocontenidos con arrays embebidos.	la actualización de catálogos. El modelo embebido de Mongo agrupa lecturas y mediciones en una sola operación de E/S de disco.
Integridad Referencial	Nativa con Claves Foráneas (FOREIGN KEY) y cascada (ON DELETE CASCADE).	Referencias lógicas mediante identificadores cruzados (lec:..., evt:...).	PostgreSQL actúa como la fuente autoritativa de la verdad. MongoDB valida tipos de datos con $jsonSchema sin bloquear tablas secundarias.
Transaccionalidad	ACID total a nivel multirregistro con aislamiento MVCC.	ACID a nivel de documento individual. Transacciones multidocumento con sobrecosto.	Las lecturas IoT son append-only; la atomicidad a nivel de documento de MongoDB ofrece máxima velocidad de ingesta concurrente.
Estructuras de Índice	Índices B-Tree estándar, compuestos y temporales.	Índices B-Tree (WiredTiger), multiclave en arrays embebidos y GeoJSON 2dsphere.	En Postgres, (id_sensor, fecha_medicion DESC) optimiza series de tiempo. En Mongo, mediciones.id_variable permite búsquedas con $elemMatch.
Expresividad de Consulta	SQL Declarativo: JOINs relacionales, subconsultas, CTEs y Window Functions.	MQL y Aggregation Framework: Canalizaciones por fases ($match, $unwind, $group).	SQL sobresale en análisis relacionales cruzados y rankings (DENSE_RANK). El Aggregation Framework sobresale en manipulación de jerarquías BSON.
Escalabilidad	Vertical principalmente; escalabilidad de lectura con réplicas.	Horizontal nativa mediante particionamiento distribuido (Sharding).	Para millones de lecturas diarias, MongoDB permite distribuir la carga entre réplicas y shards sin reestructurar el modelo.

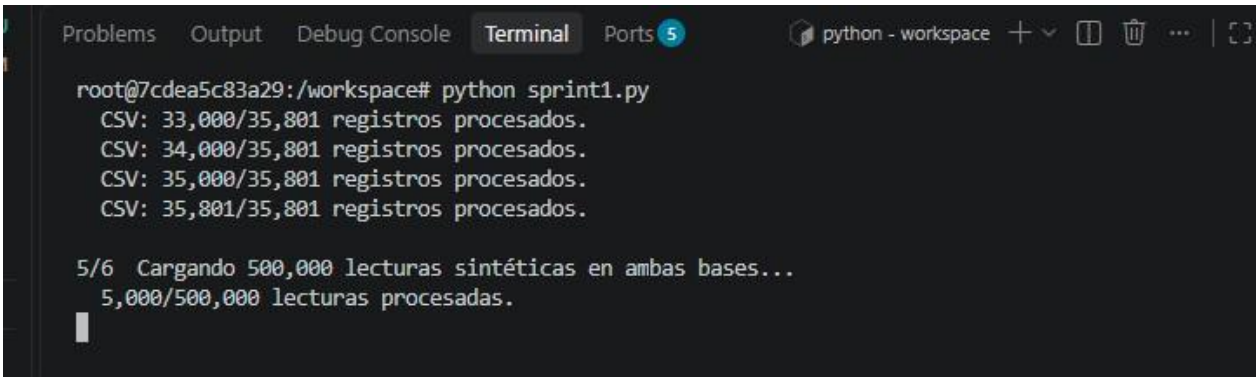
5. Evidencias y resultados

Carga de los datos

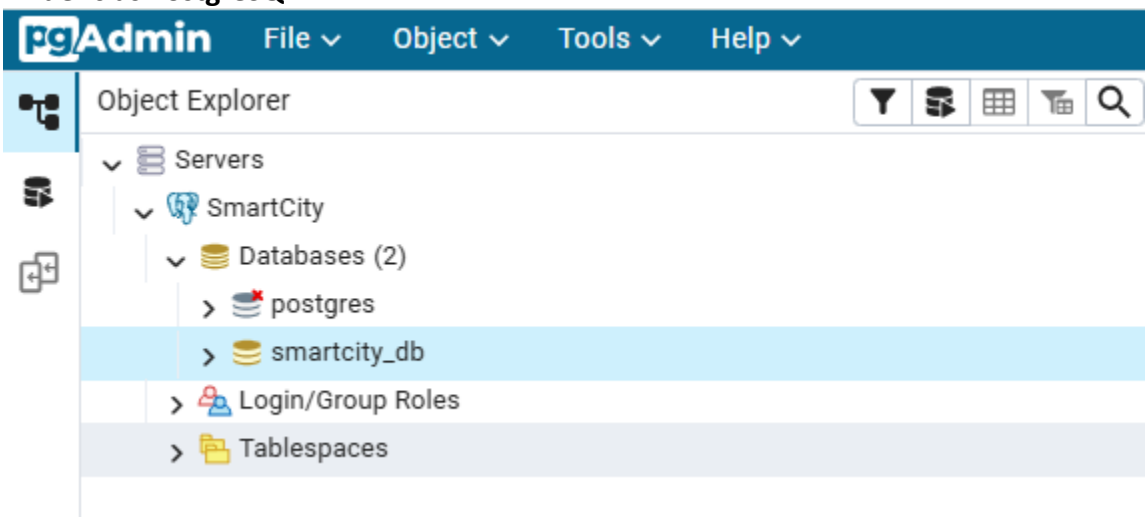
Procesamiento del CSV

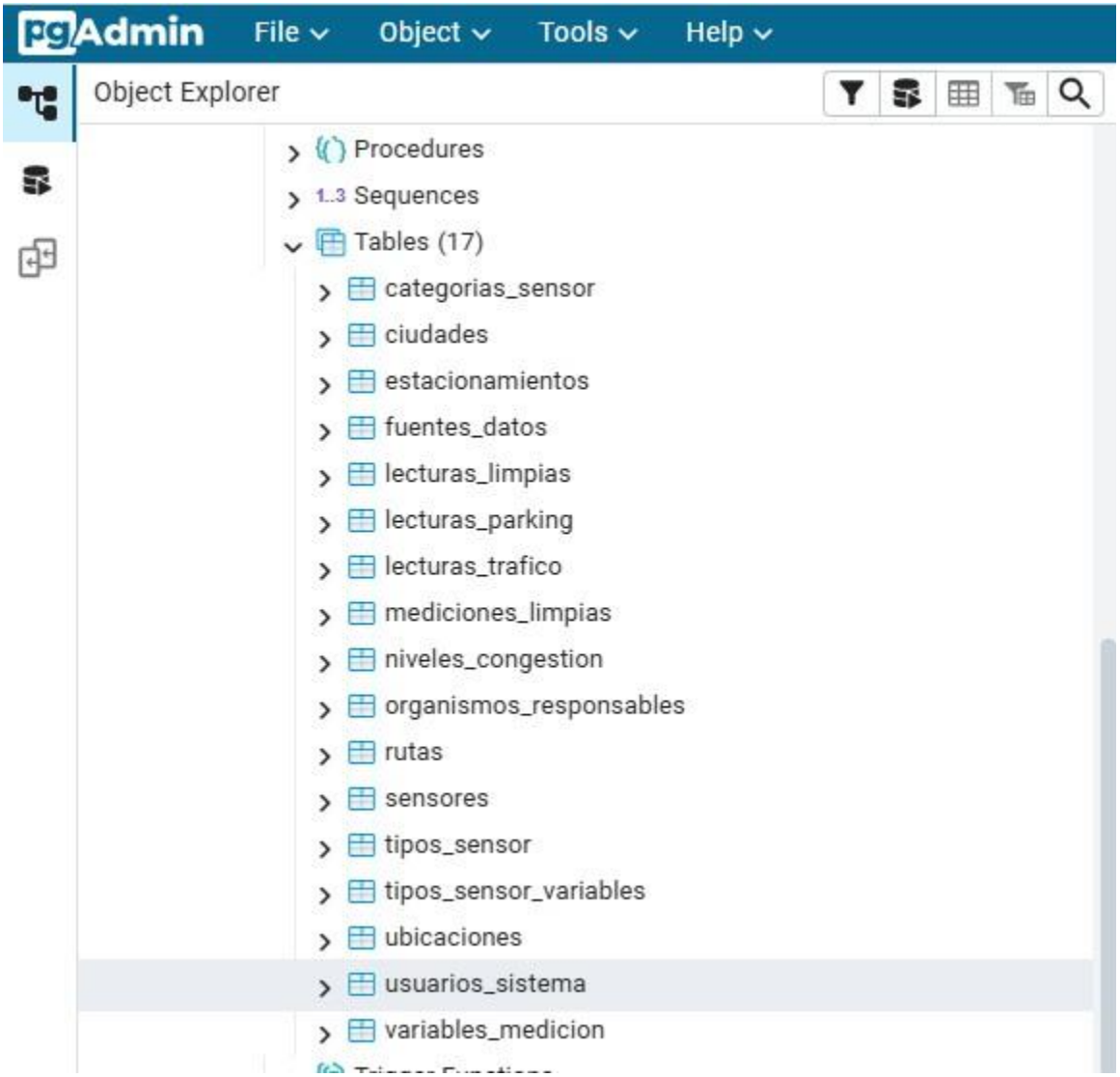


Carga de lecturas sintéticas



Evidencias PostgreSQL




pgAdmin
File
Object
Tools
Help
Object Explorer
Procedures
Sequences
Tables (17)
categorias_sensor
ciudades
estacionamientos
fuentes_datos
lecturas_limpias
lecturas_parking
lecturas_trafico
mediciones_limpias
niveles_congestion
organismos_responsables
rutas
sensores
tipos_sensor
tipos_sensor_variables
ubicaciones
usuarios_sistema
variables_medicion

Pregunta de Negocio 1: ¿Cuál zona registra mayor cantidad de vehículos?

Query | Query History

```sql
SELECT u.nombre_zona, SUM(m.valor) AS total_vehiculos
FROM mediciones_limpias m
JOIN variables_medicion v ON v.id_variable = m.id_variable
JOIN lecturas_limpias l ON l.id_lectura = m.id_lectura
JOIN sensores s ON s.id_sensor = l.id_sensor
JOIN ubicaciones u ON u.id_ubicacion = s.id_ubicacion
WHERE v.codigo_variable = 'vehiculos'
GROUP BY u.nombre_zona
ORDER BY total_vehiculos DESC
FETCH FIRST 1 ROWS ONLY;
```

	nombre_zona text	total_vehiculos numeric
1	Zona 02	850288

Pregunta de Negocio 2: ¿Cuál es la hora de mayor congestión?

Query | Query History

```sql
SELECT EXTRACT(HOUR FROM l.fecha_medicion) AS hora_del_dia,
       COUNT(*) AS lecturas_congestionadas
FROM lecturas_trafico lt
JOIN lecturas_limpias l ON l.id_lectura = lt.id_lectura
JOIN niveles_congestion nc ON nc.id_nivel = lt.id_nivel
WHERE nc.codigo = 'alta'
GROUP BY EXTRACT(HOUR FROM l.fecha_medicion)
ORDER BY lecturas_congestionadas DESC
FETCH FIRST 1 ROWS ONLY;
```

Data Output | Messages | Notifications

	hora_del_dia numeric	lecturas_congestionadas bigint
1	5	3831

Pregunta de Negocio 3: ¿Qué rutas presentan mayor utilización?

Query | Query History

```sql
SELECT r.nombre AS ruta, COUNT(lt.id_lectura) AS total_registros,
       AVG(m.valor) AS velocidad_promedio
FROM lecturas_trafico lt
JOIN rutas r ON r.id_ruta = lt.id_ruta
JOIN lecturas_limpias l ON l.id_lectura = lt.id_lectura
LEFT JOIN mediciones_limpias m ON m.id_lectura = l.id_lectura AND m.id_variable = 3
GROUP BY r.nombre
ORDER BY total_registros DESC
FETCH FIRST 5 ROWS ONLY;
```

Data Output | Messages | Notifications

SQL

	ruta text	total_registros bigint	velocidad_promedio numeric
1	Ruta ...	16667	57.1592368152636947
2	Ruta ...	16666	62.4731189247569903
3	Ruta ...	16666	62.3778351134045362
4	Ruta ...	16666	62.6072842913716549
5	Ruta 1	16666	57.6983079323172927

Evidencias MongoDB

Colecciones

Mongo Express Database: smartcity_db

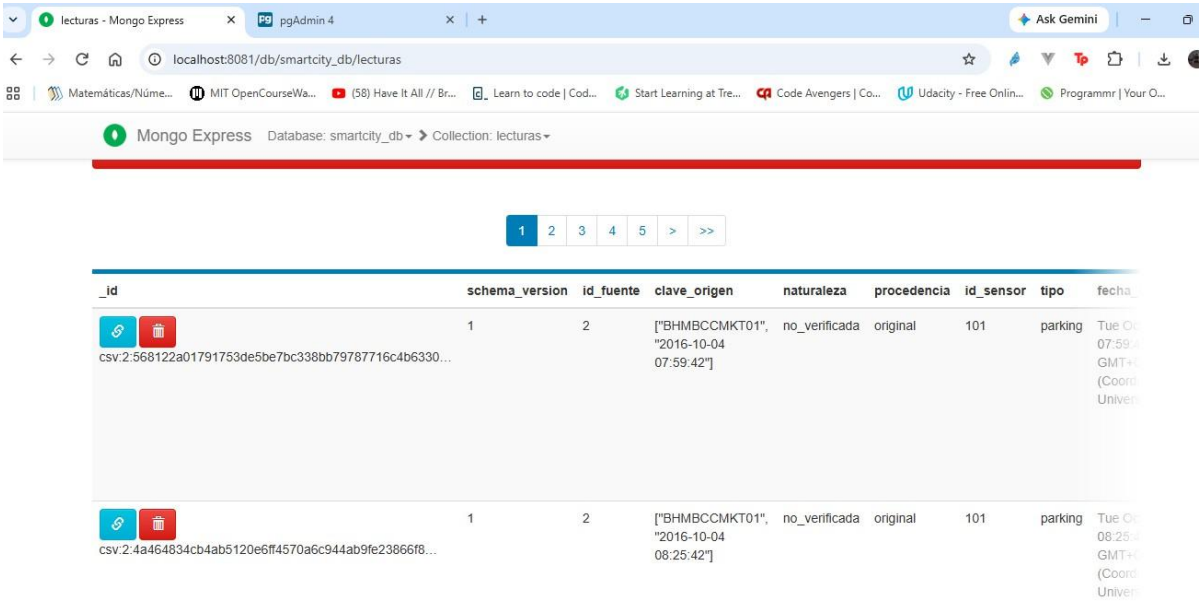
Viewing Database: smartcity_db

Collections Collection Name + Create collection

View	Export	[JSON]	Import	alertas	Del
View	Export	[JSON]	Import	crud_demo	Del
View	Export	[JSON]	Import	eventos	Del
View	Export	[JSON]	Import	incidentes	Del
View	Export	[JSON]	Import	lecturas	Del

Database Stats	
Collections (incl. system.namespaces)	5
Data Size	425 MB
Storage Size	75.0 MB
Avg Obj Size #	778 Bytes
Objects #	546305
Indexes #	20
Index Size	68.6 MB

Lecturas en MongoDB



Editing Document: lec:1:fila0000001

Back

Save



```
{
    _id: 'lec:1:fila0000001',
    schema_version: 1,
    id_fuente: 1,
    clave_origen: 'generado:fila0000001',
    naturaleza: 'sintetica',
    procedencia: 'simulado',
    id_sensor: 2,
    tipo: 'parking',
    fecha_medicion: ISODate('2025-01-01T00:01:00.000Z'),
    fecha_recepcion: ISODate('2025-01-01T00:01:10.000Z'),
    contexto: {
        id_estacionamiento: 2,
        id_ubicacion: 2,
        id_ciudad: 1,
        id_tipo_sensor: 1,
        id_categoria: 1,
        id_organismo: 2
    },
    mediciones: [
        {
            id_variable: 1,
            codigo: 'capacidad',
            valor: NumberDecimal('110'),
            unidad: 'plazas'
        },
        {
            id_variable: 2,
            codigo: 'ocupacion',
            valor: NumberDecimal('14'),
            unidad: 'plazas'
        }
    ],
    calidad: 'valida',
    raw: {
        Capacity: 110,
        Occupancy: 14
    },
    trazabilidad: {
        archivo: 'generador_sprint1',
        fila: 1,
        version_ingesta: '1.0'
    }
}
```

Back

Save

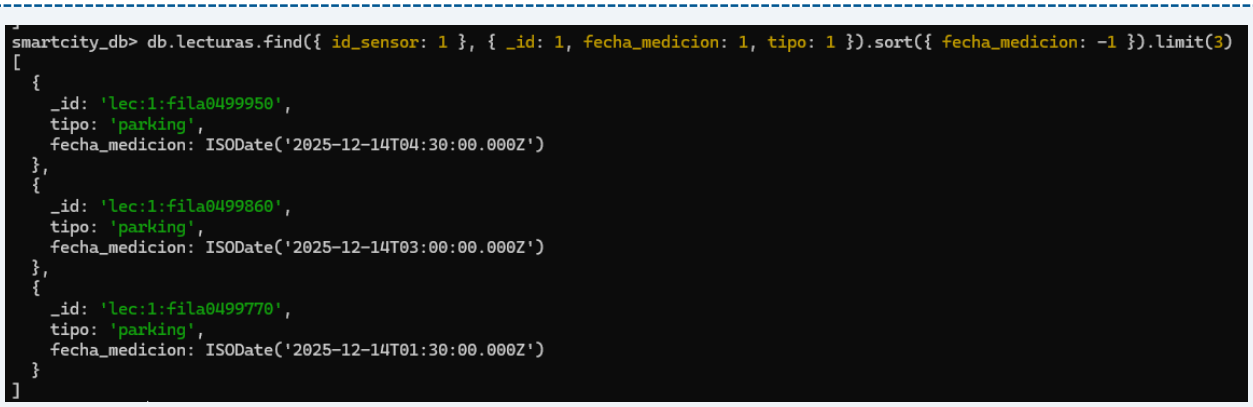
Consultas Obligatorias en MongoDB (Aggregation Framework y MQL)

De acuerdo con los requisitos del enunciado (Pág. 11 y 12), a continuación se documentan las diez consultas analíticas obligatorias implementadas en MongoDB utilizando el Aggregation Framework y filtros MQL sobre el conjunto de más de 536.000 documentos cargados en la colección 'lecturas', complementadas con la evidencia de operaciones CRUD.

Consulta MongoDB 1: Lecturas de un sensor específico

Objetivo: Filtrar la telemetría histórica de un sensor en particular ordenada cronológicamente de forma descendente, aprovechando el índice compuesto ix_sensor_fecha.

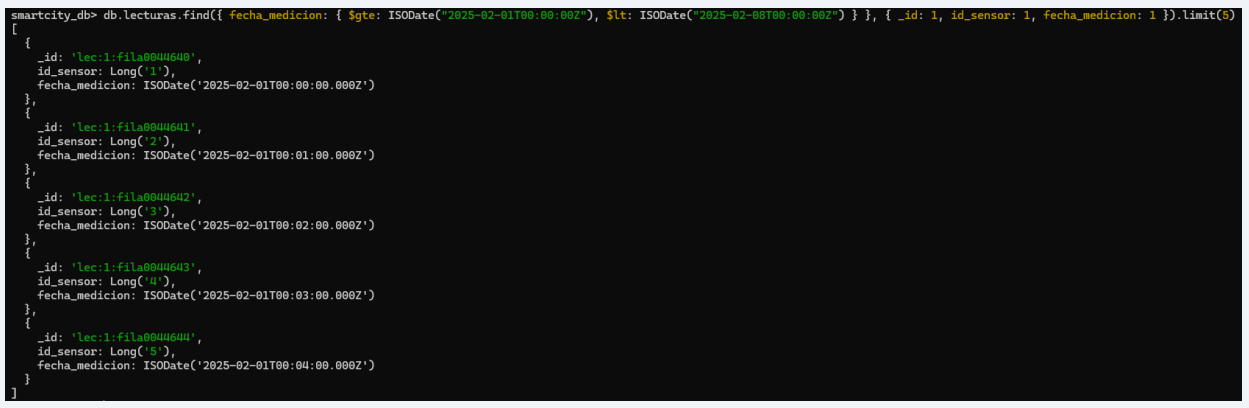
```
db.lecturas.find(
    { id_sensor: NumberLong("1") },
    { _id: 1, fecha_medicion: 1, tipo: 1, mediciones: 1 }
).sort({ fecha_medicion: -1 }).limit(3);
```



Consulta MongoDB 2: Lecturas en un rango de fechas

Objetivo: Filtrar observaciones dentro de un intervalo temporal específico mediante operadores de rango $gte y $lt sobre fechas ISO BSON indexadas.

```
db.lecturas.find({
    fecha_medicion: {
        $gte: ISODate("2025-02-01T00:00:00Z"),
        $lt: ISODate("2025-02-08T00:00:00Z")
    }
}, { _id: 1, id_sensor: 1, fecha_medicion: 1 }).limit(5);
```



Consulta MongoDB 3: Valores máximos y mínimos registrados por variable

Objetivo: Pipeline de agregación que descompone el array embebido de mediciones ($unwind) y agrupa ($group) calculando los extremos estadísticos ($min y $max) de cada variable urbana.

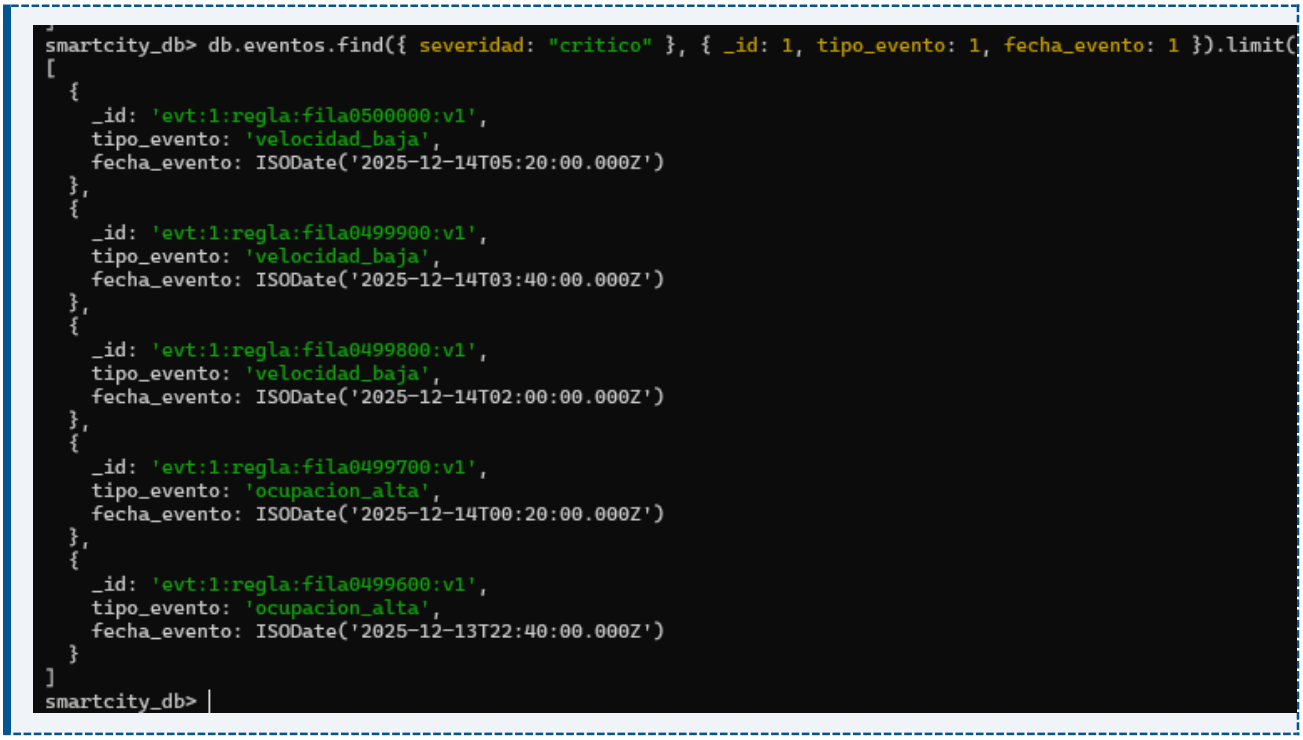
```
db.lecturas.aggregate([
    { $unwind: "$mediciones" },
    { $group: {
        _id: "$mediciones.codigo",
        valor_minimo: { $min: "$mediciones.valor" },
        valor_maximo: { $max: "$mediciones.valor" }
    }}
]);
```



Consulta MongoDB 4: Eventos críticos detectados

Objetivo: Consulta sobre la colección 'eventos' para identificar alertas de severidad crítica derivadas de anomalías en flujo vehicular y sobreocupación de parqueos.

```
db.eventos.find(
    { severidad: "critico" },
    { _id: 1, tipo_evento: 1, id_sensor: 1, fecha_evento: 1, evidencia: 1 }
).limit(5);
```

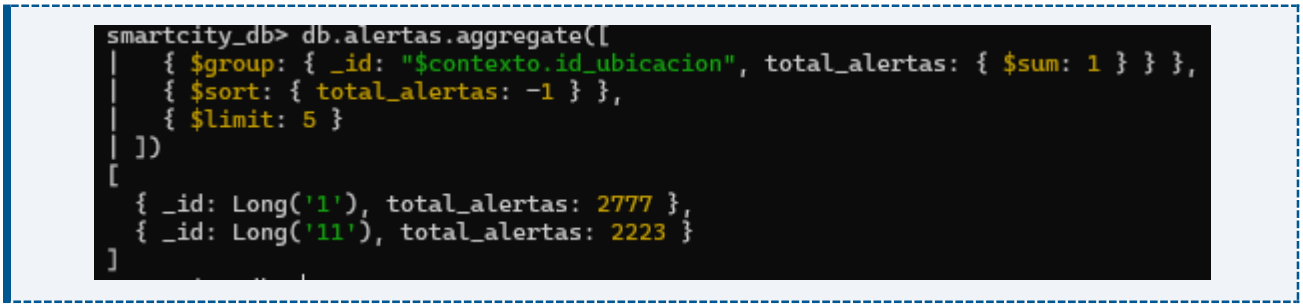


```
smartcity_db> db.eventos.find({ severidad: "critico" }, { _id: 1, tipo_evento: 1, fecha_evento: 1 }).limit(
[
  {
    _id: 'evt:1:regla:fila0500000:v1',
    tipo_evento: 'velocidad_baja',
    fecha_evento: ISODate('2025-12-14T05:20:00.000Z')
  },
  {
    _id: 'evt:1:regla:fila0499900:v1',
    tipo_evento: 'velocidad_baja',
    fecha_evento: ISODate('2025-12-14T03:40:00.000Z')
  },
  {
    _id: 'evt:1:regla:fila0499800:v1',
    tipo_evento: 'velocidad_baja',
    fecha_evento: ISODate('2025-12-14T02:00:00.000Z')
  },
  {
    _id: 'evt:1:regla:fila0499700:v1',
    tipo_evento: 'ocupacion_alta',
    fecha_evento: ISODate('2025-12-14T00:20:00.000Z')
  },
  {
    _id: 'evt:1:regla:fila0499600:v1',
    tipo_evento: 'ocupacion_alta',
    fecha_evento: ISODate('2025-12-13T22:40:00.000Z')
  }
]
smartcity_db>
```

Consulta MongoDB 5: Alertas emitidas por zona geográfica

Objetivo: Canalización de agregación sobre la colección 'alertas' agrupando por identificador de ubicación en el objeto de contexto para rankear zonas con mayor incidencia.

```
db.alertas.aggregate([
    { $group: { _id: "$contexto.id_ubicacion", total_alertas: { $sum: 1 } } },
    { $sort: { total_alertas: -1 } },
    { $limit: 5 }
]);
```



```
smartcity_db> db.alertas.aggregate([
|   { $group: { _id: "$contexto.id_ubicacion", total_alertas: { $sum: 1 } } },
|   { $sort: { total_alertas: -1 } },
|   { $limit: 5 }
| ])
[
  { _id: Long('1'), total_alertas: 2777 },
  { _id: Long('11'), total_alertas: 2223 }
]
```

Consulta MongoDB 6: Promedio de mediciones por zona y variable

Objetivo: Aggregation Framework con fases $unwind, $group jerárquico por zona geográfica y variable de medición, calculando promedios ($avg) y ordenamiento.

```
db.lecturas.aggregate([
    { $unwind: "$mediciones" },
    { $group: {
        _id: { zona: "$contexto.id_ubicacion", variable: "$mediciones.codigo" },
        promedio: { $avg: "$mediciones.valor" }
    }},
    { $sort: { "_id.zona": 1 } },
    { $limit: 10 }
]);
```



Consulta MongoDB 7: Promedio por tipo de sensor

Objetivo: Cálculo del promedio de las variables agrupadas por clase de dispositivo (sensores de estacionamiento vs. tráfico vehicular) en el contexto embebido.

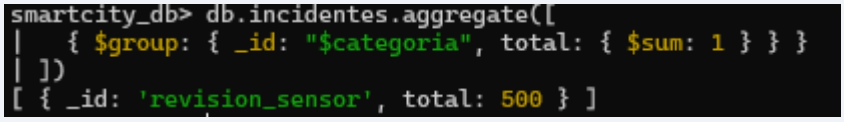
```
db.lecturas.aggregate([
    { $unwind: "$mediciones" },
    { $group: {
        _id: { tipo: "$contexto.id_tipo_sensor", variable: "$mediciones.codigo" },
        promedio: { $avg: "$mediciones.valor" }
    }}
]);
```



Consulta MongoDB 8: Incidentes urbanos registrados por categoría

Objetivo: Agrupación y conteo sobre la colección 'incidentes' para clasificar los reportes de mantenimiento y eventos de infraestructura municipal.

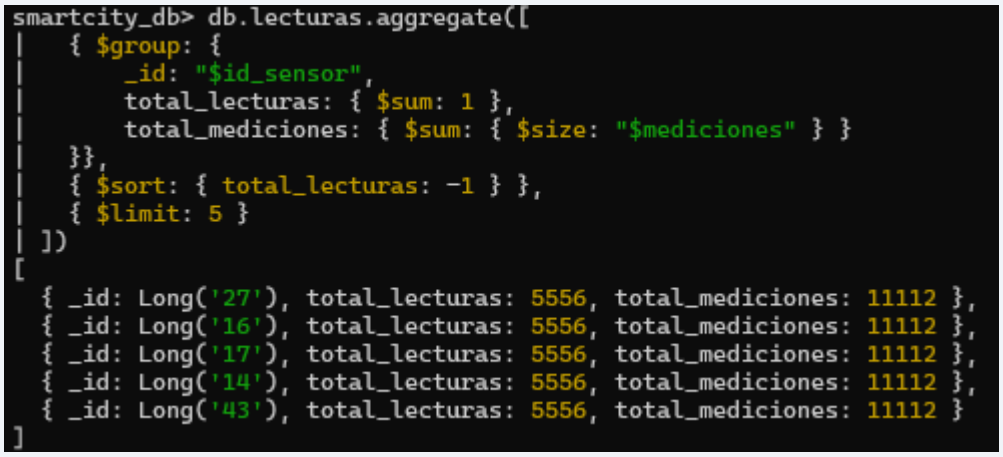
```
db.incidentes.aggregate([
    { $group: { _id: "$categoria", total_incidentes: { $sum: 1 } } },
    { $sort: { total_incidentes: -1 } }
]);
```



Consulta MongoDB 9: Total de lecturas y mediciones por sensor

Objetivo: Agregación para cuantificar el volumen de documentos generados por dispositivo y el total de mediciones individuales contenidas mediante el operador $size.

```
db.lecturas.aggregate([
    { $group: {
        _id: "$id_sensor",
        total_lecturas: { $sum: 1 },
        total_mediciones: { $sum: { $size: "$mediciones" } }
    }},
    { $sort: { total_lecturas: -1 } },
    { $limit: 5 }
]);
```



Consulta MongoDB 10: Filtros múltiples sobre arrays anidados ($elemMatch)

Objetivo: Búsqueda compuesta que valida simultáneamente atributos del documento raíz, claves de contexto de ubicación y condiciones sobre el array de mediciones embebidas mediante $elemMatch.

```
db.lecturas.find({
    id_sensor: NumberLong("1"),
    "contexto.id_ubicacion": NumberLong("1"),
    mediciones: {
        $elemMatch: {
            id_variable: NumberLong("2"),
            valor: { $gt: NumberDecimal("50") }
        }
    }
}, { _id: 1, fecha_medicion: 1, mediciones: 1 }).limit(3);
```

```
smartcity_db> db.lecturas.find({
|   id_sensor: 1,
|   "contexto.id_ubicacion": 1,
|   mediciones: { $elemMatch: { id_variable: 2, valor: { $gt: 50 } } }
| }, { _id: 1, fecha_medicion: 1, mediciones: 1 }).limit(3)
[
  {
    _id: 'lec:1:fila0499860',
    fecha_medicion: ISODate('2025-12-14T03:00:00.000Z'),
    mediciones: [
      {
        id_variable: Long('1'),
        codigo: 'capacidad',
        valor: Decimal128('105'),
        unidad: 'plazas'
      },
      {
        id_variable: Long('2'),
        codigo: 'ocupacion',
        valor: Decimal128('81'),
        unidad: 'plazas'
      }
    ]
  },
  {
    _id: 'lec:1:fila0499680',
    fecha_medicion: ISODate('2025-12-14T00:00:00.000Z'),
    mediciones: [
      {
        id_variable: Long('1'),
        codigo: 'capacidad',
        valor: Decimal128('105'),
        unidad: 'plazas'
      },
      {
        id_variable: Long('2'),
        codigo: 'ocupacion',
        valor: Decimal128('99'),
        unidad: 'plazas'
      }
    ]
  },
  {
    _id: 'lec:1:fila0499500',
    fecha_medicion: ISODate('2025-12-13T21:00:00.000Z'),
    mediciones: [
      {
        id_variable: Long('1'),
        codigo: 'capacidad',
        valor: Decimal128('105'),
        unidad: 'plazas'
      },
      {
        id_variable: Long('2'),
        codigo: 'ocupacion',
        valor: Decimal128('105'),
        unidad: 'plazas'
      }
    ]
  }
]
```

Demostración de Operaciones CRUD en MongoDB (Colección crud_demo)

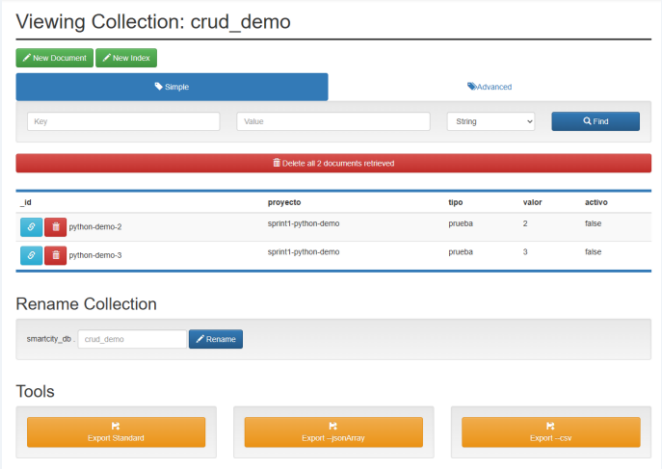
Objetivo: Comprobación obligatoria de las operaciones de manipulación documental: insertOne, insertMany, find, findOne, updateOne, updateMany y deleteOne en una colección dedicada de prueba.

```
// 1. insertOne e insertMany
db.crud_demo.insertOne({ _id: "demo-1", proyecto: "sprint1", valor: 10, activo: true });
db.crud_demo.insertMany([
    { _id: "demo-2", proyecto: "sprint1", valor: 20, activo: true },
    { _id: "demo-3", proyecto: "sprint1", valor: 30, activo: true }
]);

// 2. findOne y find
db.crud_demo.findOne({ _id: "demo-1" });
db.crud_demo.find({ proyecto: "sprint1" }).sort({ valor: 1 });

// 3. updateOne y updateMany
db.crud_demo.updateOne({ _id: "demo-1" }, { $set: { valor: 99 } });
db.crud_demo.updateMany({ proyecto: "sprint1" }, { $set: { activo: false } });

// 4. deleteOne
db.crud_demo.deleteOne({ _id: "demo-1" });
```



Conclusiones

El diseño asigna a PostgreSQL la organización de catálogos y relaciones, y a MongoDB el almacenamiento documental de las observaciones y su contexto. Las claves compartidas permiten rastrear los registros entre motores, mientras que las restricciones y validaciones permiten controlar inconsistencias durante la carga.

Repo del proyecto GitHub

https://github.com/Allan19Prieto/smartcity_project